PyTorch CONFERENCE EUROPE 2026

Model Compression API for a fast and precise inference in ExecuTorch

Daniil Lyakhov Aamir Nazir @ Intel Corporation

intel.

Abstract

We present a **Model Compression API** for ExecuTorch, powered by the **Neural Network Compression Framework (NNCF)**. It extends the standard `torch.ao` quantization pipeline with data-aware algorithms — `quantize_pt2e` for post-training quantization and `compress_pt2e` for weight compression.

We demonstrate that the model compression with the **NNCF API** yields significant speedups on Llama 3.2, YOLOv12, and Stable Diffusion while preserving target metrics — for the **OpenVINO ExecuTorch backend**.

The NNCF compression API operates on the quantization annotations provided by a given `torch.ao` Quantizer — rather than on backend-specific internals — so any vendor backend can leverage its data-aware algorithms without altering the compressed model architecture.

NNCF Compression API

NNCF's `quantize_pt2e` and `compress_pt2e` extend the standart torch.ao quantization pipeline with data-aware algorithms while preserving compatibility with the `torch.ao` Quantizer.

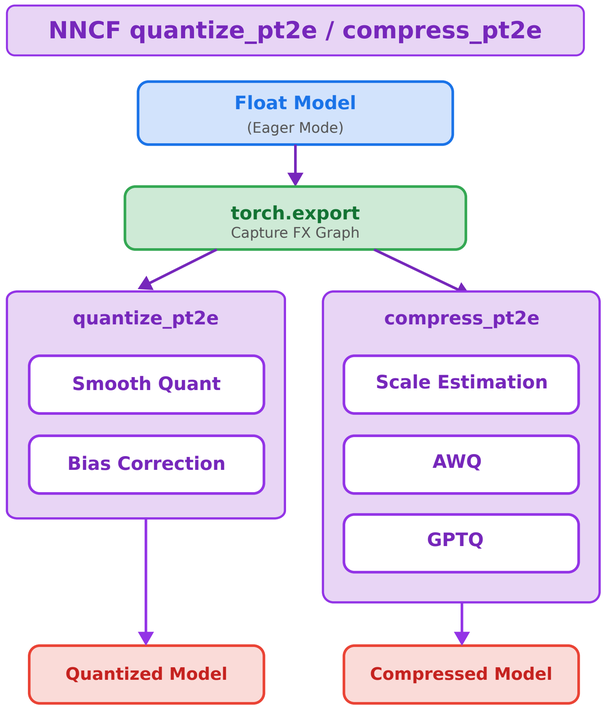

Figure 1. NNCF `quantize_pt2e` / `compress_pt2e`.

Weight Compression — `compress_pt2e`

Applies weight compression to a `torch.fx.GraphModule`, targeting LLM deployment. Key algorithms:

- **Scale Estimation** — Estimates optimal quantization scales for 4-bit layers to further reduce perplexity of compressed LLMs.

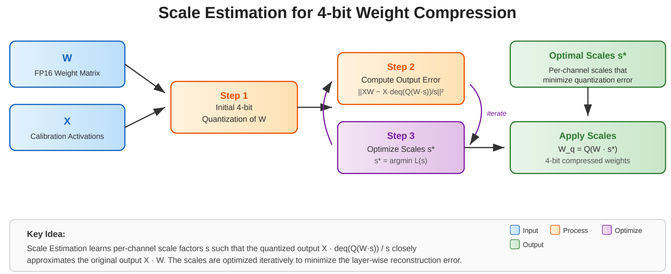

Figure 2. Visual scheme of Scale Estimation algorithm.

- **AWQ** — Activation-aware Weight Quantization that finds optimal per-channel scales to minimize quantization error based on activation distributions [6].
- **GPTQ** — Applies the GPTQ algorithm for accurate layer-wise weight quantization using second-order information [4].
- **LoRA Correction** — Applies Low-Rank Adaptation corrections to compensate for weight compression errors [5].

Currently, `compress_pt2e` supports the `OpenVINOQuantizer` and the OpenVINO backend. Support for third-party quantizers can be added by implementing a lightweight NNCF quantizer adapter.

Post-Training Quantization — `quantize_pt2e`

Can be applied on top of *any* `torch.ao` Quantizer to improve the accuracy of the quantized models. The API implements the standard `prepare_pt2e`/`convert_pt2e` flow and augments it with data-aware algorithms:

1. **SmoothQuant** — Reduces activation quantization error by inserting smoothing scales before weighted layers [9].

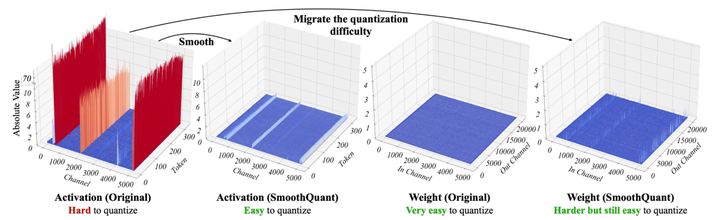

Figure 3. Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant.

2. **BiasCorrection** — Corrects bias shifts introduced by quantization [7].

This means existing quantization pipelines can benefit from NNCF's accuracy-boosting algorithms without changing the quantizer — simply replace the standard `torch.ao` quantization call with `quantize_pt2e` and supply a calibration dataset.

Results

LLM: Llama 3.2 Weight Compression

Model / Mode	Perplexity ↓	Model Size
Llama 3.2 1B (FP16)	—	—
Llama 3.2 1B (INT8)	—	—
Llama 3.2 1B (INT4 + Scale Est.)	—	—
Llama 3.2 3B (FP16)	—	—
Llama 3.2 3B (INT8)	—	—
Llama 3.2 3B (INT4 + Scale Est.)	—	—

Table 1. Llama 3.2 perplexity and model size under different compression modes.

Detection: YOLOv12 INT8 Quantization

INT8 post-training quantization applied via `quantize_pt2e` with SmoothQuant and Bias-Correction [3].

Model / Mode	mAP@50 ↑	mAP@50:95 ↑	Model Size
YOLOv12 (FP32)	—	—	—
YOLOv12 (INT8)	—	—	—

Table 2. YOLOv12 detection accuracy: FP32 baseline vs. INT8 quantized.

Image Generation: Stable Diffusion

Mixed-precision pipeline: INT8 quantization for U-Net via `quantize_pt2e`, weight compression for text encoder and VAE decoder via `compress_pt2e` [2].

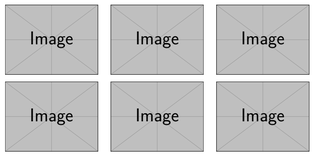

Figure 4. Stable Diffusion generation quality: FP32 (top) vs. INT8/compressed (bottom) for three prompts.

References

[1] Llama 3.2 int8/int4 compression with scale estimation for executorch. `https://github.com/pytorch/executorch/pull/16002`, 2025.

[2] Stable diffusion int8 quantization and weight compression for executorch. `https://github.com/pytorch/executorch/pull/17807`, 2025.

[3] Yolov12 int8 quantization example for executorch. `https://github.com/pytorch/executorch/tree/main/examples/models/yolo12`, 2025.

[4] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. `https://arxiv.org/abs/2210.17323`, 2022.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. `https://arxiv.org/abs/2106.09685`, 2021.

[6] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. `https://arxiv.org/abs/2306.00978`, 2023.

[7] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. Data-free quantization through weight equalization and bias correction. `https://arxiv.org/abs/1906.04721`, 2019.

[8] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[9] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. `https://arxiv.org/abs/2211.10438`, 2022.